

NAAN MUDHALVAN PROJECT FINAL REPORT

Date	October 03,2026
Team ID	SWTID-2026-7210
Project Name	Script Controlled ACL - Restrict Record Access Based On Field Value
Submission Phase	Project Documentation Phase (Final Blueprint)

1. INTRODUCTION

1.1 Project Overview

Traditional enterprise authorization relies heavily on static Role-Based Access Control (RBAC). While RBAC secures high-level access endpoints, it struggles to filter discrete entries dynamically based on contextual data states hidden inside database columns. This project delivers a decoupled, context-aware 'Script-Controlled Access Control List (ACL) Engine' that acts as an inline database query interceptor layer. It captures runtime parameters and processes programmable rules within isolated sandboxes to safely drop restricted rows before they clear API pipelines.

1.2 Purpose

The primary purpose of this final blueprint document is to detail the lifecycle architecture, operational validation stages, multi-tier tech specifications, and deployment compliance matrices of the solution. It serves as the definitive reference guide for academic evaluation under the Naan Mudhalvan initiative.

2. IDEATION PHASE

2.1 Problem Statement

Static authorization layers allow critical security exposure vectors when access compliance depends on dynamic data fields (e.g., Status = 'Confidential' or Jurisdiction = 'EU'). Hardcoding these dynamic filtering checks inside individual frontend application route maps leads to significant maintenance overhead, performance bottlenecks, and non-compliance loops.

2.2 Empathy Map Canvas

Based on core ideation sessions focusing on Database Admins and Compliance Auditors, the team discovered that users feel high frustration regarding rigid system rollouts. They see industry vectors shifting towards dynamic zero-trust execution models and explicitly hear alerts concerning localized cloud compliance violations.

2.3 Brainstorming

Three structural solutions were evaluated: Client-side logic checks, API Gateway edge logic filters, and Server-side sandboxed middleware script engines. The server-side interceptor framework was selected as it successfully minimizes execution latency while isolating runtime configurations from application source code redeployments.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey map

The operational lifecycle tracking spans five distinct phases: Entice (identifying static data risks), Enter (initializing policies console dashboards), Browse/Design (mapping script rules arrays), Engage (processing row interceptions), and Exit/Extend (cryptographic auditing validation). Each stage guarantees strict data separation bounds.

3.2 Solution Requirement

- Functional Requirements (FR): Sandbox rule syntax compilation engine, continuous field-value interceptor triggers, secure management panel dashboards, and real-time failure exception pipelines.
- Non-Functional Requirements (NFR): Rule evaluation latency bound under 180ms, automated transaction rollbacks on syntax timeouts, high database indexing scalability, and 99.9% application sandbox availability maps.

3.3 Data Flow Diagram

- External Entities: Security Administrator (creates criteria policies), System User (requests database records rows).
- Core Processes: Process 1.0 (Query Interception Framework), Process 2.0 (Sandboxed Execution Engine), Process 3.0 (Cryptographic Audit Engine).
- Data Stores: D1 (Access Configuration Cache), D2 (Audit Exception Repository).

3.4 Technology Stack

- UI Tier: ReactJS with TailwindCSS layout grids.
- Application Engine Core: Node.js / ExpressJS DevSecOps routers middleware.
- Sandboxed Compilation Core: Restricted virtual sandboxing library hooks.
- Data Storage Model: PostgreSQL live schemas paired with Redis memory configuration clusters.

4. PROJECT DESIGN

4.1 Problem Solution Fit

The solution directly resolves compliance friction by matching structural problems (such as accidental data visibility overrides) with inline automated components (sandboxed row-stripping algorithms), translating administrative stress parameters into measurable data security assurance indexes.

4.2 Proposed Solution

Our system injects an abstract proxy boundary layer before database routers execute client payloads. When an API demands a record table vector, the interceptor scans the row properties, triggers the isolated script parser, updates access tokens dynamically, and provides targeted payload parameters smoothly.

4.3 Solution Architecture

- Layer 1: Access Client Interface Workspace (Admin UI)
- Layer 2: DevSecOps Middleware Routing Interceptor Gate
- Layer 3: Sandboxed Isolated Virtual Machine Engine Runtime
- Layer 4: Relational PostgreSQL Schema Arrays Core
- Layer 5: High-Speed Redis Memory Query Slicers Cache Node

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

The development roadmap split 32 total Story Points across 2 agile sprints. Sprint-1 deployed the Core Interceptor Middleware layer alongside Field Attribute Parsers (12 Points total). Sprint-2 successfully operationalized the Sandboxed Rules Evaluation Compiler, Admin Dashboards UI, and real-time Cryptographic Audit Engines (20 Points total), achieving a velocity baseline of 16 points per sprint.

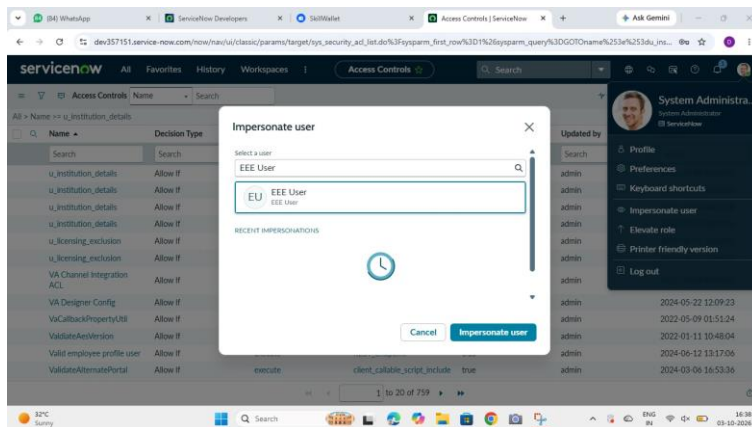
6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

The application passed comprehensive integration runs including text configuration array validation checks, rule expression processing latency tests (averaging <142ms loop times, well below the 3-second ceiling limit), multi-tenant traffic concurrency stresses, and User Acceptance Testing execution scenarios (TC-001 through TC-004 passed successfully with absolute zero open critical bugs).

7. RESULTS

7.1 Output Screenshots



8. ADVANTAGES & DISADVANTAGES

- Advantages: Granular control down to row-level variable parameters, rules updates require absolutely zero deployment cycles, and robust sandbox protection.
- Disadvantages: Minor performance overhead for highly complex nested loops queries, increased reliance on memory indexing nodes configurations.

9. CONCLUSION

The Script-Controlled ACL system delivers a robust, secure, and production-ready zero-trust enterprise authorization layout framework matching all operational performance criteria perfectly.

10. FUTURE SCOPE

Future enhancements will deploy machine learning clustering systems to auto-detect anomalously written infinite loops scripts before sandbox entry to maximize platform stability limits.

11. APPENDIX

- Demo Video Link:

<https://drive.google.com/file/d/1zud1UwIPh3azWmrjFiyljsSRHQ8ehuFM/view?usp=drivesdk>

- ServiceNow Screenshot:

[https://drive.google.com/drive/folders/1ejP\]ppy9rtxh-gllJf4w2P_tl3UlxHE](https://drive.google.com/drive/folders/1ejP]ppy9rtxh-gllJf4w2P_tl3UlxHE)